

1 IGEMM 系列算子优化手册

1.1 IGEMM 系列算子背景

1.1.1 应用背景

在 DeepSeek 团队开发的面向 DeepSeekV3/R1 等模型的推理系统中，INT8 GEMM(IGEMM)的核心功能为执行 INT8 量化激活值与 INT8 量化权重的打包和其间的矩阵乘法并反量化输出 BF16。其具体的应用场景主要包括：

1. MLA 模块中 qkv_a/q_b/o 函数；
2. MLP 模块中 MLP 门控函数，MoE 模块中共享专家/混合专家门控函数；

1.1.2 设计层次背景

INT8 GEMM(IGEMM) 系列算子功能可概括为三个功能层次，层次由低级到高级，为 920F 面向多种场景提供高性能计算的灵活性：

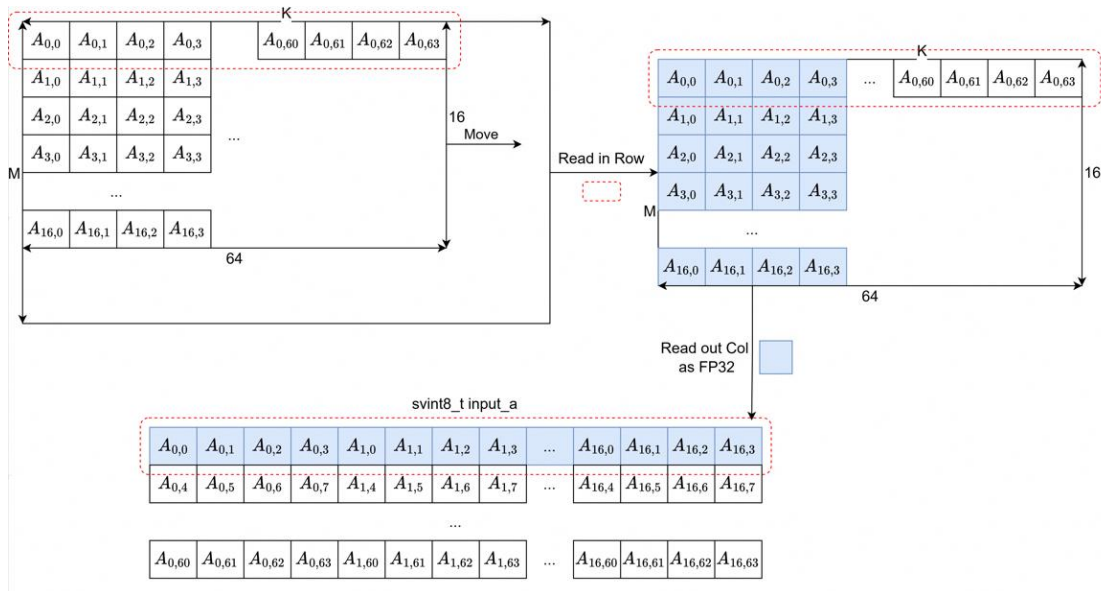
1. Kernel 层：其功能主要是对指定数据块执行单一功能的基础操作。主要包括 SME 友好的数据打包 Pack Kernel，针对 920F 有显著优化的 Prefetch Pipeline Kernel，和旨在实现输出快速反量化的 BF16 Dequant Save Kernel(BDQ)等；
2. Thread 层：其功能是基于单线程的矩阵语义计算任务，为上层调用提供简洁的矩阵操作抽象，任务模块由 Kernel 组合，可针对不同场景设计不同的线程计算任务，主要包括 igemm_bdq(dynamic)_thread_task, , gemm_pack_thread_task 等；
3. Schedule 层：其功能是基于多线程角度对整体计算任务进行线程拆分，通过对 Thread 层操作进行不同策略的任务封装，从而实现对不同计算场景的快速高性能调优，主要包括 igemm_pack, igemm_bdq, igemm_bdq_dynamic_tile_n, dual_experts_parallel 等；

1.2 IGEMM 系列算子基本原理

1.2.1 Pack Kernel 与 mopa 指令基本原理

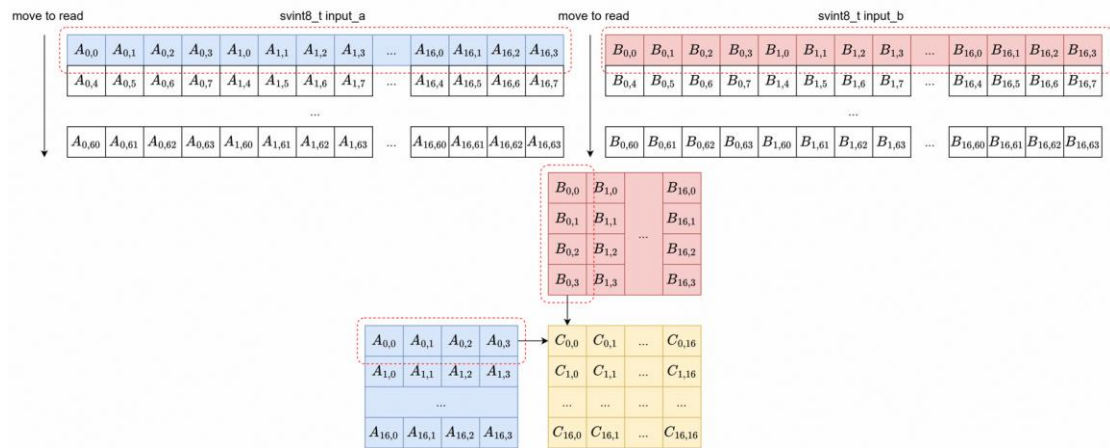
SME 指令集通过 ZA 二维向量寄存器实现输入矩阵快速 Pack 和基于外积原理对 Pack 矩阵执行矩阵乘法操作。对于 IGEMM 系列算子，具体过程如下：

1. 对于输入矩阵 $[M, K]$ ，pack kernel 以 $[16, 64]$ 块为步进以行主序进行 pack，如图表 1 所示，对于单个 $[16, 64]$ 块，逐行读入 ZA tile，完整读入 $[16, 64]$ INT8 矩阵后以 FP32 规格从 ZA tile 逐列读出，输出一个 $[16 * 64]$ 的数组，此即为 $[16, 64]$ 的 pack 输出。



图表 1 Pack Kernel 原理

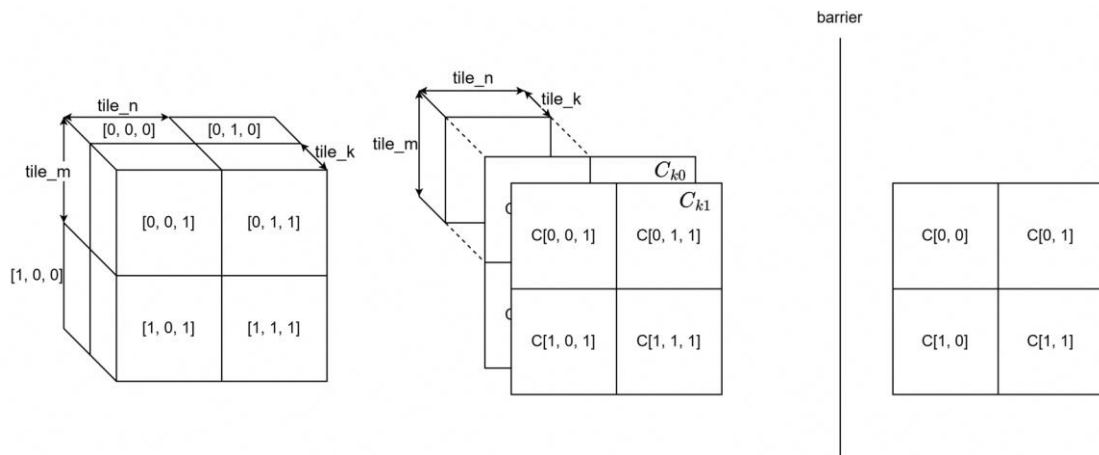
2. 对于已完成 Pack 的左矩阵 $[M, K]$ 与右矩阵 $[N, K]$ ，则通过 mopa 指令进行快速矩阵乘，如图表 2 Pack 矩阵访存顺序和 MOPA 指令原理所示，通过顺序连续访存 Pack 矩阵，MOPA 对输入向量得以执行外积规约操作，由于 Pack 过程中 INT8 实际是以 FP32 格式逐列读出的，因此每一次 MOPA 实际上是做了一次 $[M, N, K] = [16, 16, 4]$ 的 INT8 矩阵乘法， C_{ij} 为 FP32，考虑 INT8 场景下 ZA 寄存器为 4 tile, 因此 ZA 寄存器一次可计算 $[64, 16, 4]$ 或 $[32, 32, 4]$ INT8 GEMM。



图表 2 Pack 矩阵访存顺序和 MOPA 指令原理

1.2.2 分块基本原理

实际的计算任务往往需要通过分块切分将计算任务拆解为多个子任务。计算过程如下图所示，我们不妨将计算任务 $\text{IGEMM}[M, N, K]$ 看做是长方体，则长方体分块则对应子任务，若 $K \neq \text{tile}_k$ ，则需要多线程 barrier 保证所有 C_{ki} 均完成计算，随后通过 reduce 规约为最终结果，如下图所示：



图表 3 IGEMM 分块基本计算过程

1.3 IGEMM 系列算子优化方案

1.3.1 线程内切分方案

a) 高 L2 HIT 的微内核线程内切分方案

在单个线程视野内，每个线程需要计算线程所属的[tile_m, tile_n, tile_k]的 IGEMM 矩阵乘法计算。在tile_m, tile_k较大的场景下，Cache 的 Miss Rate 会快速升高，这意味着每一次计算所需的数据都需要从内存中获取，这对内存带宽造成了较大的访存压力。因此，需要设计一种能够在 Cache 中保留一定数据，以此减小内存访存带宽压力的方法。

Roofline 建模分析：

微内核就是在单线程得到任务切分[tile_m, tile_n, tile_k]的基础上进一步切分计算任务为串行计算多个[micro_m, micro_n, micro_k] IGEMM，以尝试提升 CPU Cache HIT Rate。以下为基于 Roofline 建模的过程：

1. 在 Roofline 模型中，公式 $\text{Machine Balance} = \frac{\text{Peak OPS}}{\text{Peak Bandwidth}}$ ，可以推算不同访存层级的 Machine Balance 值

Machine Balance 理论值：

- a) HBM: $\frac{960\text{TOPS}}{4\text{TB/s}} = 240;$
- b) L2 Cache: $\frac{2048\text{ OP}}{64\text{ Byte}} = 32;$
- c) L1 Cache: $\frac{2048\text{ OP}}{2 * 64\text{ Byte}} = 16;$

Machine Balance 实测值：

- a) HBM: $\frac{960\text{TOPS} / 8}{320\text{GB/s}} = 375;$
- b) L2 Cache: $\frac{960\text{TOPS} / 304}{60\text{ GB/s}} = 52.63;$
- c) L1 Cache: $\frac{960\text{TOPS} / 304}{100\text{ GB/s}} = 31.57;$

2. 基于下述原因：

a)
$$\text{IGEMM Roofline} = \frac{2 * 16 * 16 * 4 \text{ OP}}{64 \text{ Byte}} = \frac{2048 \text{ OP}}{64 \text{ Byte}} = 32$$
, 因此 IGEMM 在 HBM 层

和 L2 Cache 为访存密集型计算, L2 Cache 访存压力显著小于 HBM。而在 L1 Cache 为计算密集型计算;

- b) 同时注意到 L1 Cache 容量极小, 无法支撑 SME 计算模式所需的最低计算窗口所需内存容量, 而 L2 Cache 容量更大, 存在提高命中率的优化空间;

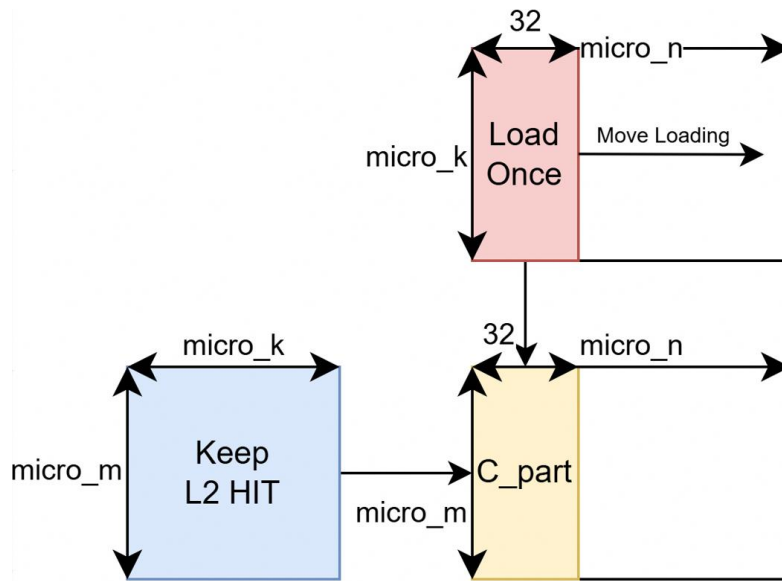
优化方向确定为减小 HBM 访存, 并提高 L2 Cache 的命中率。在 Roofline 上表现为将计算由 L2 Roofline 的访存密集区向 L2 Cache Roofline 的计算密集区转移。

计算模式设计: 将[micro_m, micro_n, micro_k]拆分为多个[micro_m, 32, micro_k]计算窗口串行计算。

理论基础:

1. 经实验测试, 通过将换入 L2 Cache 的计算访存规模约束在 L2 Cache 大小约 40%以下。可以保持该部分数据在 L2 Cache 的高 HIT Rate。
2. 对于预期不常使用的一次性数据如[32, micro_k], 通过 load non-template 指令, 可以使其访存不流经 CPU Lx Cache。
3. 由于 L2 Cache 换入其他数据的可能性减小, 使预期反复使用的数据如[micro_m, micro_k]在一定程度上避免被 L2 Cache 换出, 提高了 HIT Rate。

访存模式: 经实验验证, 通过将左矩阵[micro_m, micro_k]规模约束在, $\text{micro_m} \leq 128$ 且 $\text{micro_k} \leq 2560$, 并同时在加载右矩阵[32, micro_k]的过程中使用 non-template 指令即 ldnt, 可以使[micro_m, micro_k]保持在 L2 Cache。实际访存仅在计算第一个[micro_m, 32, micro_k] 时需要从内存中获取[micro_m, micro_k], L2 只需要替换[32, micro_k] 即可推进计算, 此时计算过程中[micro_m, micro_k] 均保持在 L2 缓存中持续命中, 如下图所示。



图表 4 [micro_m, micro_k] L2 HIT 示意图

寄存器级 Roofline 建模:

对已有 IGEMM 计算目标 $[M, N, K]$, $[tile_m, tile_n, tile_k]$, 可通过逼近下述非线性优化目标最大值以计算最优 $[micro_m, micro_n, micro_k]$:

寄存器级Roofline =

$$\frac{2 * tile_m * tile_n * tile_k}{\frac{tile_k}{micro_k} * \frac{tile_m}{micro_m} * (micro_m * micro_k + tile_n * micro_k + micro_m * tile_n * 2)} = \frac{2 * tile_m * tile_n * tile_k}{(tile_m * tile_k + tile_n * tile_k * \frac{tile_m}{micro_m} + tile_m * tile_n * \frac{tile_k}{micro_k} * 2)}, micro_m * micro_k <$$

Threshold = 768KB * (40 ± 5)%

1. Decode 侧样例:

a) qkva. igemm_bdq $[M, N, K] = [64, 264, 7168]$, 已知 $[tile_m, tile_n, tile_k] = [64, 132, 448]$ 。

i. 尝试最小化过程 Roofline, 已知 $tile_m * tile_k = 28KB < Threshold$, 则令 $micro_m = M$, $micro_k = K$ 。此时 $Roofline =$

$$\frac{2 * 64 * 132 * 448}{64 * 448 + 132 * 448 + 64 * 132 * 2} = 72.29, \text{ 此时 Roofline 在策略中最大}$$

ii. 此时计算过程为 $[M, N_i] = [64, 32]$ 作为计算窗口逐渐移动, 最终完成 $[M, N] = [64, 132]$ 的线程任务计算。在此期间 $[M, K] = [64, 448]$ 常驻 L2 Cache, 每次 $[M, N_i] = [64, 32]$ 仅需要 HBM 读取 $[N, K]^1 = [32, 448]$ 即可。

注 1: 这里的[N, K]同[M, K]只是输入矩阵 Pack 后输出规模的一种表示, 由于 SME 对左右矩阵 Pack 后以[16, tile_k]为基本组成单元, 这里的表述不表示实际形状。相比较非 SME 外积计算单元执行的 GEMM[M, N, K]要求输入矩阵形状严格为[M, K]和[K, N], 当前 SME IGEMM Pack 要求 Pack 输入左右矩阵形状为[M, K]和[N, K]。

- b) o. igemmbdq [M, N, K] = [64, 7168, 2048], 已知[tile_m, tile_n, tile_k] = [64, 224, 2048]
- i. 尝试最小化过程 Roofline, 已知 $tile_m * tile_k = 128KB < Threshold$, 则令 $micro_m = M$, $micro_k = K$ 。此时 $Roofline = \frac{2 * 64 * 224 * 2048}{64 * 2048 + 224 * 2048 + 64 * 224 * 2} = 94.94$, 此时同样 Roofline 在策略中最大
- ii. 此时计算过程为[M, N_i] = [64, 32]作为计算窗口逐渐移动, 最终完成[M, N] = [64, 224]的线程任务计算。在此期间[M, K] = [64, 2048]常驻 L2 Cache, 每次[M, N_i] = [64, 32]仅需要 HBM 读取 [N, K] = [32, 2048]即可。

2. Prefill 侧样例:

- a) o. igemmbdq [M, N, K] = [4096, 7168, 2048], 已知 [tile_m, tile_n, tile_k] = [4096, 224, 2048]
- i. 尝试最小化过程 Roofline, 已知 $tile_m * tile_k = 8192KB > Threshold$, 则令 $micro_m * micro_k = 256K$ 则求解得到 $micro_m = 128$, $micro_k = K$ (由于优化解空间为非线性空间, 建议在解析解附近多次实验以获取最优值, 这里数据直接由理论和实验给出)。此时 $Roofline = \frac{2 * 128 * 224 * 2048}{128 * 2048 + 224 * 2048 * \frac{4096}{128} + 128 * 224 * 2} = 150.90$, 此时 Roofline 值最大。
- ii. 则单线程计算目标为[M, N] = [4096, 224], 此时为保证 L2 Cache 能够容纳左矩阵, 则需切分micro_m为最大 128, 则过程为首先由 [M_i, N_j] = [128, 32]作为计算窗口逐渐移动, 首先完成移动窗口

$[M_i, N] = [128, 224]$, 然后最终完成 $[M, N] = [4096, 224]$ 的线程任务计算。在此期间每一组 $[M_i, N] = [128, 224]$ 的计算过程 $[M, K] = [128, 2048]$ 常驻 L2 Cache, 每次 $[M_i, N_j] = [128, 32]$ 仅需要 HBM 读取 $[N, K] = [32, 2048]$ 即可。

b) Prefetch Pipeline Kernel

920F 执行 INT8 GEMM 所需的 INT8 MOPA 指令速度快于 FP32/BF16, 这导致 INT8 GEMM 的访存压力大幅增加, 如何提高计算访存掩盖程度成为提升 INT8 GEMM 计算效率的核心目标。

理论基础:

1. **MOPA/Prefetch 指令掩盖率推演:** 考虑 2.0GHz 场景下, 理论推演和实际测试均可得到结论: 8 条 MOPA 的执行延时和 1 条 Prefetch 指令的访存延时可以实现计算访存掩盖, 以下为推演过程:

- a) **理想场景:** 在 Stream 访存测试中测得访存带宽为 400GB/s, 耗时 $\frac{64\text{Byte} * 32}{400\text{GB/s}} = 5.12\text{ns}$, 而在指令延时和吞吐实验中测得 INT8 MOPA 延时 3.5cycle, 吞吐为 1.14 inc/cycle, 因此 8 条 INT8 MOPA 的理想延时为 $\frac{8}{1.14} + 3.5 = 10.5\text{cycle}$, 约合 $10.5 * \frac{1\text{s}}{2.0\text{GHz}} = 5.25\text{ns}$ 。注意到 $5.12\text{ns} \approx 5.25\text{ns}$, 因此在理想场景中认为实现了计算访存掩盖;

- b) **实际测试:** 基于理想场景的结论设计的 Prefetch Pipeline Kernel 在 Decode 侧的不同 IGEMM 算子中 GEMM 过程的内存带宽利用为 320GB/s~360GB/s, 因此 $\frac{64\text{Byte} * 32}{320 \sim 360\text{GB/s}} = 5.68 \sim 6.4\text{ns}$, 此时 8 条 INT8 实际的吞吐为 $\frac{8}{(5.68 \sim 6.4\text{ns}) * 2.0\text{GHz} - 3.5} = 0.86 \sim 1.01\text{ inc/cycle}$, 对比 4 条 INT8 MOPA 和 12 条 INT8 MOPA 的吞吐影响, 判定 8 条 INT8 MOPA 和 1 条 Prefetch 相互掩盖确为最优;

- c) **气泡占比:** $\frac{1.14 - (0.86 \sim 1.01)}{1.14} = 11.4 \sim 24\%$; 推测气泡产生原因为:

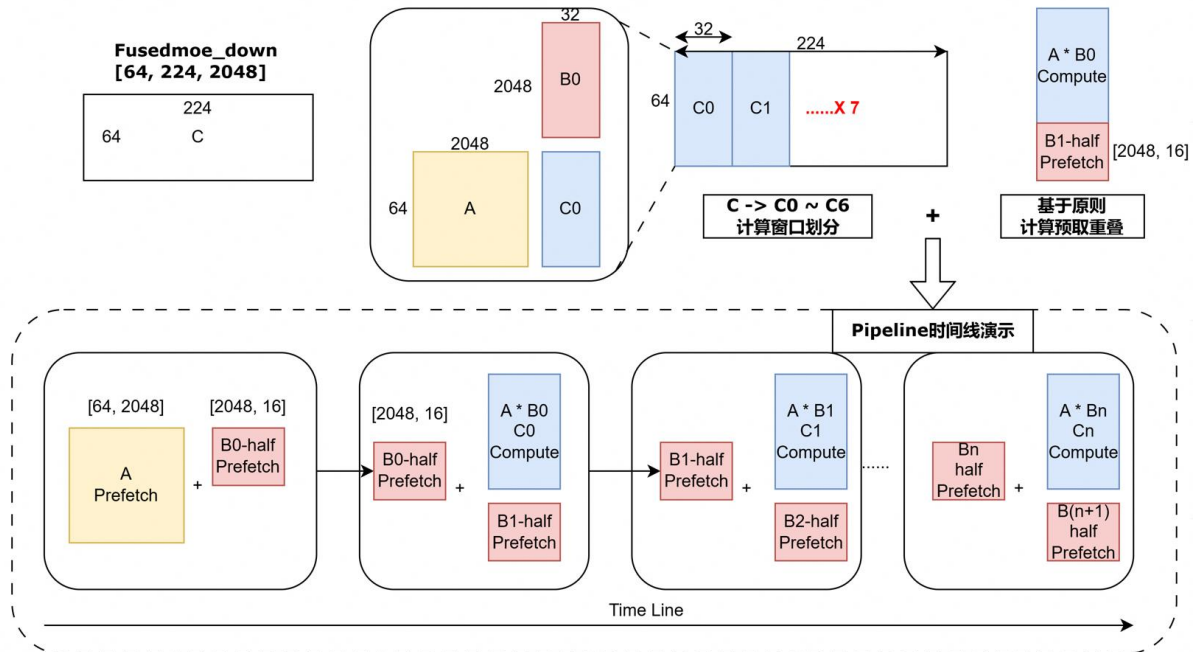
- i. L2 Cache 预期外换出引发的 Cache Miss 导致;
 - i. MOPA 指令和 Load 指令间存在短程依赖, 而 Load 指令的访存延时可能存在波动, 进而影响流水线整体利用率。
2. 因此对于 $\text{IGEMM}[M, N, K]$, 对于一个计算窗口 $[M, 32, K]$, 在 $M\%32 = 0$ 时需要 MOPA 指令数为 $\frac{M}{32} * \frac{32}{32} * \frac{K}{4} * 4 = \frac{M*K}{32}$ 。此时可以掩盖 Prefetch 指令 $\frac{M*K}{32} * \frac{1}{8} = \frac{M*K}{256}$ 条, 可访存 $\frac{M*K}{4}$ 字节, 由于计算窗口中右矩阵大小为 $[32, K]$, 因此对于不同 M 大小, 一个计算窗口能够预取的右矩阵大小也不同。在 $M = 128$ 时, 计算和访存能够完全相互掩盖, $M < 128$ 时只能部分掩盖, 优先保证内存带宽的高利用率。

基于该规律, DeepSeek 应用优化团队设计了 Prefetch Pipeline Kernel, 旨在减少冗余访存和减少访存 miss 导致的 MOPA 停车。

计算流程: 如图表 5 Prefetch Pipeline Kernel 工作原理演示。过程重点在于在计算窗口之间构建 Prefetch 流水线, 首先通过第一个计算窗口输入完全预取到 L2 实现流水线初始化, 以保证 HBM 带宽完全用于下一个计算窗口的右矩阵预取。因此在 GEMM 过程中插入 Prefetch 指令, 以保证 MOPA 计算指令和下一个计算窗口右矩阵的 HBM 访存相互掩盖。

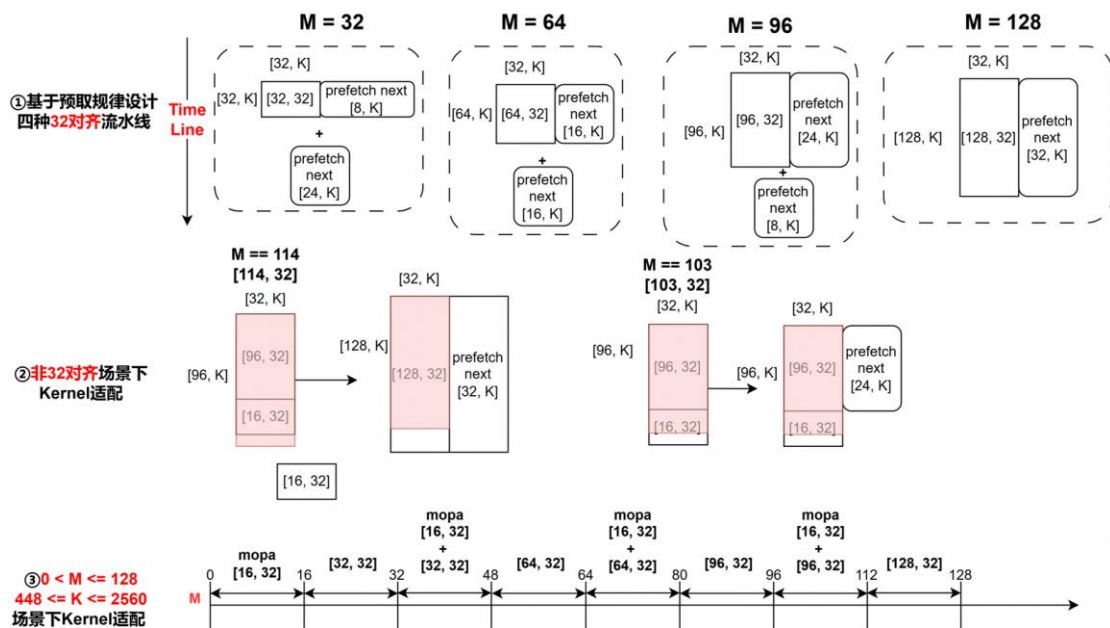
对如图所示的 $M = 64$ 场景, 根据理论基础可计算得到单个计算窗口可以预取下一个计算窗口的 50%。随后进入下一个计算窗口, 将未预取的部分完整预取后重复上述流程。通过调整 Prefetch 指令的插入规律, 能够实现 MOPA 指令

执行与 HBM 预取访存最大程度相互掩盖。实现了 Decode 侧的最优计算效率。



图表 5 Prefetch Pipeline Kernel 工作原理演示

流水线架构推广：在构建如上图所示的流水线基本结构后，DeepSeek 团队结合高 L2 HIT 的微内核切分方案和 Prefetch Pipeline Kernel，基于理论基础可以计算不同 M 下预取下一个计算窗口的比例，从而推广流水线结构到 $M \in [0, 128], K \in [448, 2560]$ 的任意微内核计算规模上，如下图所示：



图表 6 流水线在不同计算规模的推广

1.3.2 线程间切分方案：动态规模下的动态切分 Pack/GEMM

计算任务引入分块机制来进行任务切分的过程如下：

线程间切分参数的初始化和设计：

1. 线程间切分应当提供两点保证：

- a) 对于各种 Shape 矩阵，切分后各线程计算量均衡；
- b) 线程 Tile Shape 应当对提高 SME 计算效率友好；

2. 保证一：切分后各线程计算量均衡；

- a) 通过 GEMM 阶段 $\frac{N}{\text{tile_n}} * \frac{K}{\text{tile_k}} = \text{thread_num}$ 公式来组合设定 tile_n , tile_k , 通过对 $[M, N, K]$ 按照 $[M, \text{tile_n}, \text{tile_k}]$ 进行切分 (为支持流水线高性能 Kernel, 现强制要求 $M = \text{tile_m}$), 可以将计算任务拆解为多个线程子任务 $[M, \text{tile_n}, \text{tile_k}]$, 计算量均衡

3. 保证二：线程 Tile Shape 应当对提高 SME 计算效率友好；

- a) SME 计算效率友好的 Shape 特征：

在满足微内核切分和 Prefetch Pipeline Kernel 的前提下, SME 对 $\text{tile_m}(M)\%128 \rightarrow 0$, tile_n 尽可能大, $448 \leq \text{tile_k} \leq 2560$, 此时 SME 效率最高

- b) 面向上述特征的最优切分：

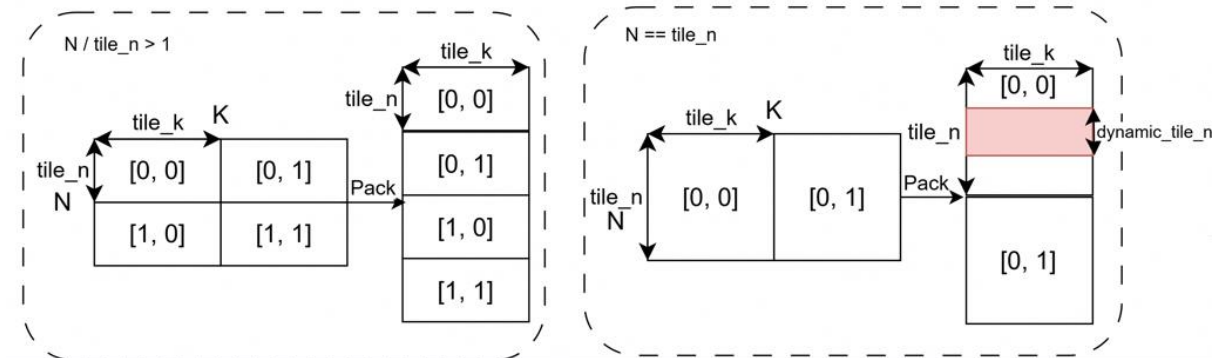
- i. 对满足 $\text{tile_m}(M)\%128 = 0 \parallel \text{tile_m}(M) < 128$ 的一般矩阵, 直接遵循规则 $\text{tile_m} = M$, tile_n 尽可能大, $448 \leq \text{tile_k} \leq 2560$ 即可保证最高效率。
- ii. 对于 $\text{tile_m}(M)\%128 \neq 0 \&\& \text{tile_m}(M) > 128$ 的异形矩阵, 需在确定 $448 \leq \text{tile_k} \leq 2560$ 的基础上测试多种 $[\text{tile_m}, \text{tile_n}]$ 的实机性能以选取最优切分。

Pack 阶段线程间切分参数选择：左矩阵按照 $[M, \text{tile_k}]$, 右矩阵则按照 $[\text{tile_n}, \text{tile_k}]$ 或 $[N, \text{tile_k}]$, Pack 阶段 tile 只影响输出格式, 不影响 Pack 执行线程数量。由于左矩阵已按照 $[M, \text{tile_k}]$ 执行 Pack, 考虑右矩阵 (权重矩阵) 的 N 维度的两种分块：

1. 若右矩阵按照[tile_n, tile_k] Pack, 则如图所示, tile_n * tile_k为一个连续块, 之后的 GEMM 阶段将严格按照设定 tile 执行, 如图表 7

$\frac{N}{tile_n} > 1$ 与 $N = tile_n$ Pack 输出对比左图所示。

2. 若右矩阵按照[N, tile_k]Pack, 则可以注意到此时N * tile_k为 Pack 连续块, 此时N * tile_k可在计算过程中动态划分为多个连续访存的子 dynamic_tile_n * tile_k, 和既有的[M, tile_k]的动态划分组合, 可以在 GEMM 执行时基于实际的 M 维度大小调整 tile。由于在推理系统中, 激活值 Pack 晚于 INT8 权重 Pack, 而混合专家激活值和 Prefill 时激活值的 M 维度往往是随机的, 可以在 M 较大时通过拆tile_m补tile_n提高计算效率, 如图表 7 $\frac{N}{tile_n} > 1$ 与 $N = tile_n$ Pack 输出对比右图所示。因此称为 dynamic_tile Pack 优化。

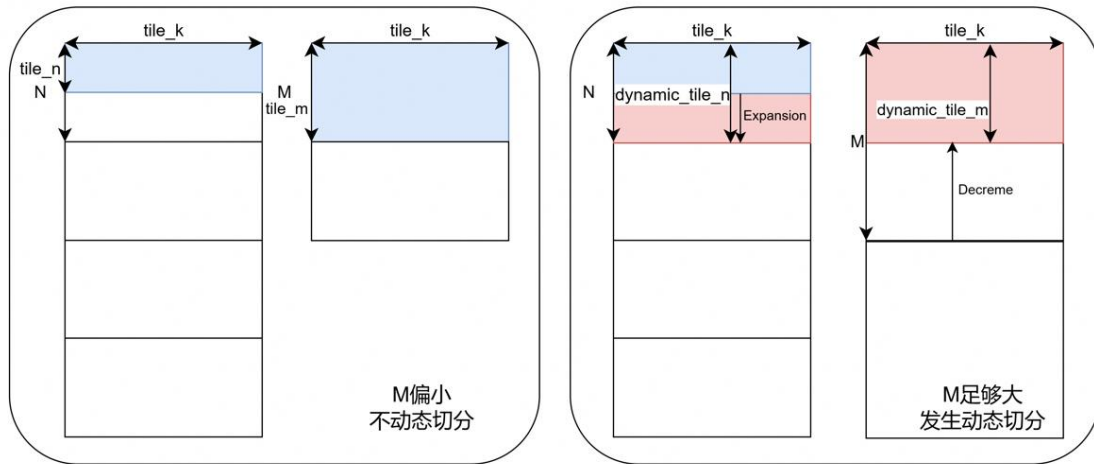


图表 7 $\frac{N}{tile_n} > 1$ 与 $N = tile_n$ Pack 输出对比

GEMM 线程间切分参数选择: GEMM 实际计算过程中所依照的 tile 分块, 基于 Pack 阶段的 tile 参数选择, 也分为两类:

1. 同 Pack 阶段 1, GEMM 阶段将严格按照设定 tile 执行, 无动态调整。
2. 同 Pack1 阶段 2, 由于左右矩阵分别按照[M, tile_k]和[N, tile_k]执行切分, 因此实际 Pack 输出中分别以M * tile_k和N * tile_k为连续块, 这为混合专家中随机 Token 分发导致的 M 维度未知这一场景提供了动态切分优化的可能。
 - a) 若 M 较小, 此时切分 M 将引起计算效率大幅下滑, 因此动态切分机制不工作, GEMM 按照设定 tile 切分, 如下图左图所示。

- b) 若 M 足够大 ($M \geq 256$) 时, M 将执行分块切分, 通过将当前左矩阵的计算范围由 $[M, \text{tile}_k]$ 分割为 $[\text{dynamic_tile}_m, \text{tile}_k]$, 右矩阵的计算范围从而可以由 $[\text{tile}_n, \text{tile}_k]$ 扩张为 $[\text{dynamic_tile}_n, \text{tile}_k]$, 如下图右图所示。 M 足额时分割, 计算效率发生微弱下降, N 扩张后, 计算效率有显著提升, 因此总体计算效率提升。



图表 8 动态切分示意图

1.3.3 Quant Save Pack 融合算子

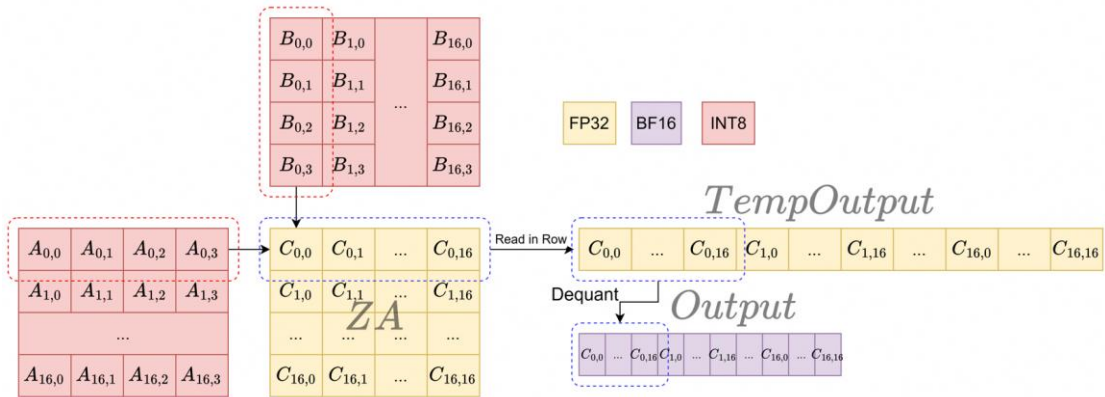
在 DeepSeek 应用中存在需要将 BF16 激活值先 Quant 量化为 INT8 激活值随后执行 Pack 的需求。具体过程包括, 首先量化 BF16 矩阵 $[M, K]$ 为 INT8 矩阵, 随后执行 Pack 到 INT8 输出矩阵。

在一般串行计算过程中, 先 Quant 后 Pack 的访存包含 Quant 阶段 $[M, K]$ 输入 BF16 矩阵, $[M, K]$ 输出 INT8 矩阵, 以及 Pack 阶段的 $[M, K]$ 读取 INT8 矩阵 (L2 Cache 未命中), $[M, K]$ 写入 INT8 矩阵, 累计 $M * K * (\text{sizeof}(\text{BF16}) + 3 * \text{sizeof}(\text{INT8}))$ 。

而若融合 Quant Save Pack, 在读取 BF16 向量时原地量化为 INT8 向量, 随后直接存入 ZA 寄存器后按 Pack 格式存入输出 INT8 矩阵, 此过程访存包含 $[M, K]$ 输入 BF16 矩阵, $[M, K]$ 输出 INT8 矩阵, 累计 $M * K * (\text{sizeof}(\text{BF16}) + \text{sizeof}(\text{INT8}))$ 。访存量为融合前的 $\frac{M * K * (\text{sizeof}(\text{BF16}) + \text{sizeof}(\text{INT8}))}{M * K * (\text{sizeof}(\text{BF16}) + 3 * \text{sizeof}(\text{INT8}))} = 0.6$ 倍。具有显著的访存优化。

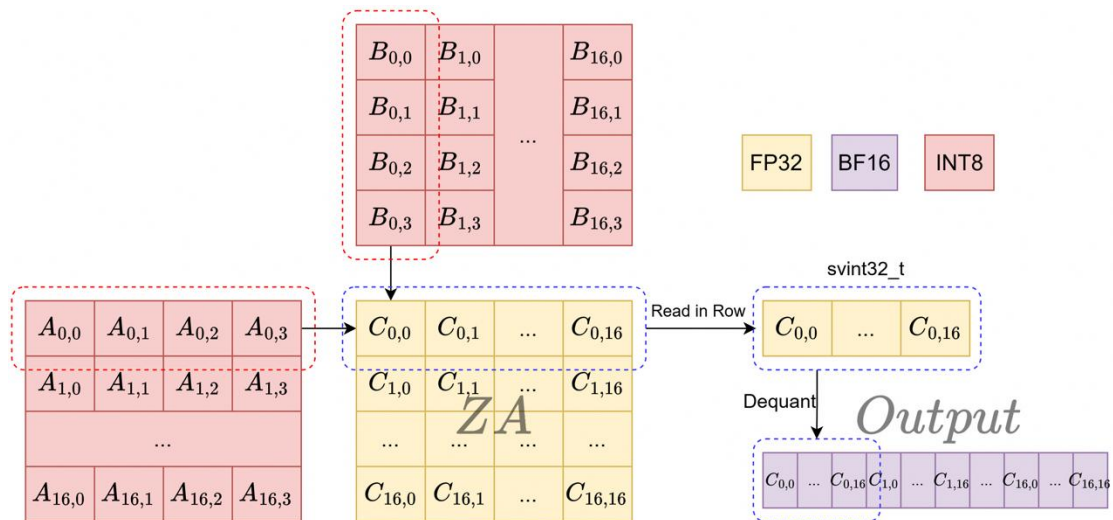
1.3.4 Output Dequant in time

如章节 1.2.1 所示，INT8 GEMM 在 ZA 寄存器 tile 的实际输出为 FP32 格式，而目标输出的格式为 BF16。因此需要反量化操作将 FP32 量化为 BF16。通过将该过程与 save 操作融合，可以大幅减小访存量并提高计算效率。



图表 9 Output Save 和 Dequant 未融合

如上图所示，在 save 操作与 Dequant 未融合时，首先需要将 FP32 输出存入 $\text{TempOutput}[M, N]$ ，然后执行 Dequant 存入 $\text{Output}[M, N]$ 。过程需访存 $M * N * (2 * \text{sizeof}(\text{FP32}) + \text{sizeof}(\text{BF16}))$ 。



图表 10 Output Save 和 Dequant 融合

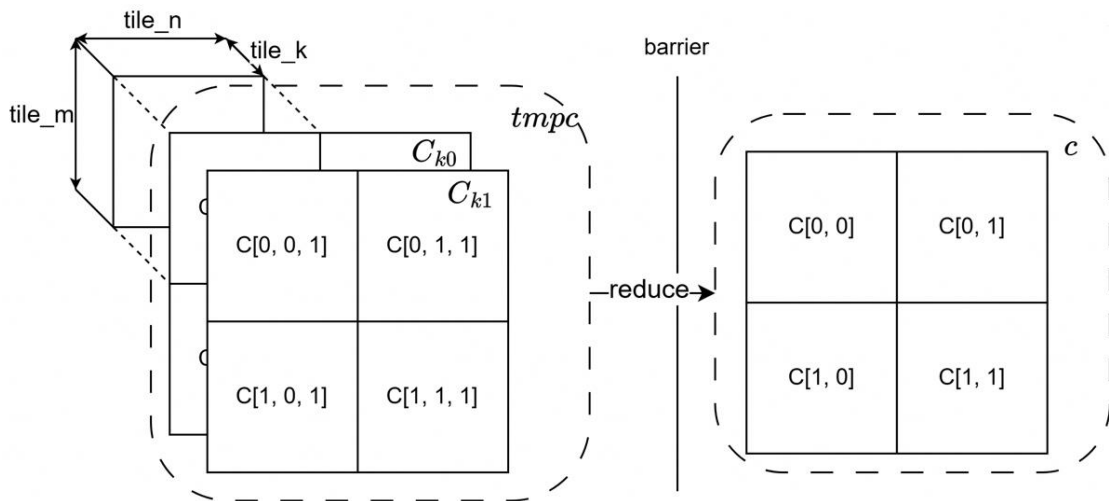
如上图所示，在 save 操作和 Dequant 操作融合后，通过临时变量获取 FP32 输出，Dequant 到 BF16 后直接存入 $\text{Output}[M, N]$ 。过程需访存 $M * N *$

$sizeof(BF16)$ 。访存量降低至原来的 $\frac{M*N* sizeof(BF16)}{M*N*(2* sizeof(FP32)+ sizeof(BF16))} = \frac{1}{5}$ 。具有显著收益。

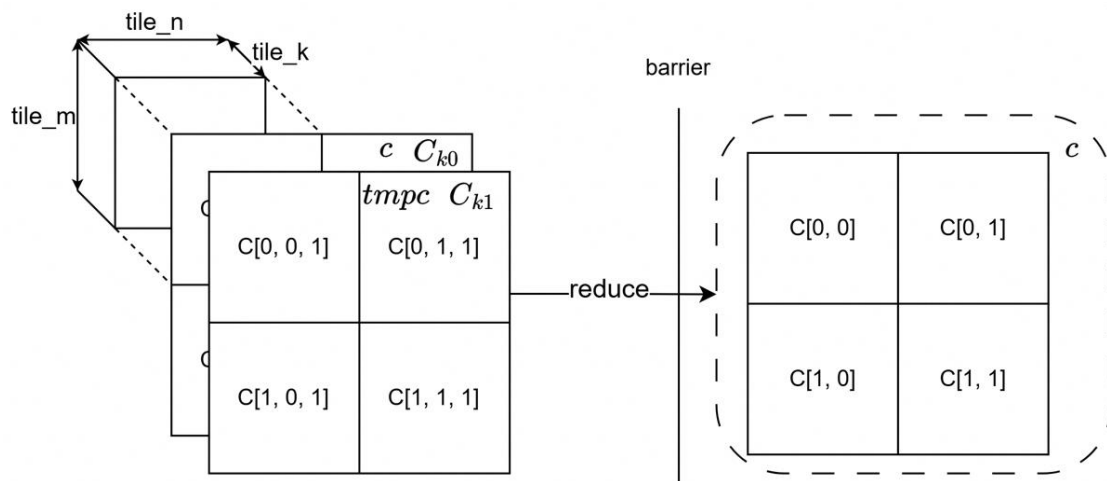
1.3.5 Reduce Optimization

对于 $K \neq \text{tile_k}$ 的场景，也即应用 K Slice 计算方法时，可通过令 $\text{idx_k} = 0$ 的输出位置由临时工作区改为输出矩阵 C，由此减少一次 $m * n$ 的矩阵访存。

如图所示，原有归约过程访存量为 $(\frac{K}{\text{tile_k}} + 1) * m * n$ ，而现有归约过程访存量为 $\frac{K}{\text{tile_k}} * m * n$ ，经实验测试，该优化方案对于 $\frac{K}{\text{tile_k}} \leq 4$ 的场景有显著优化。



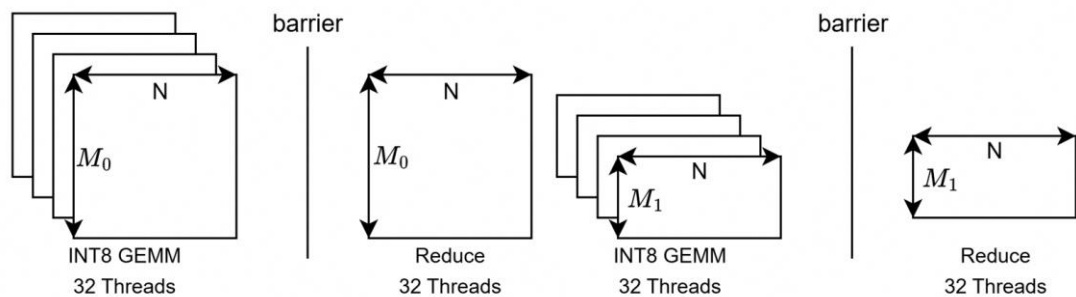
图表 11 K Slice 原归约方案



图表 12 K Slice 现有归约方案

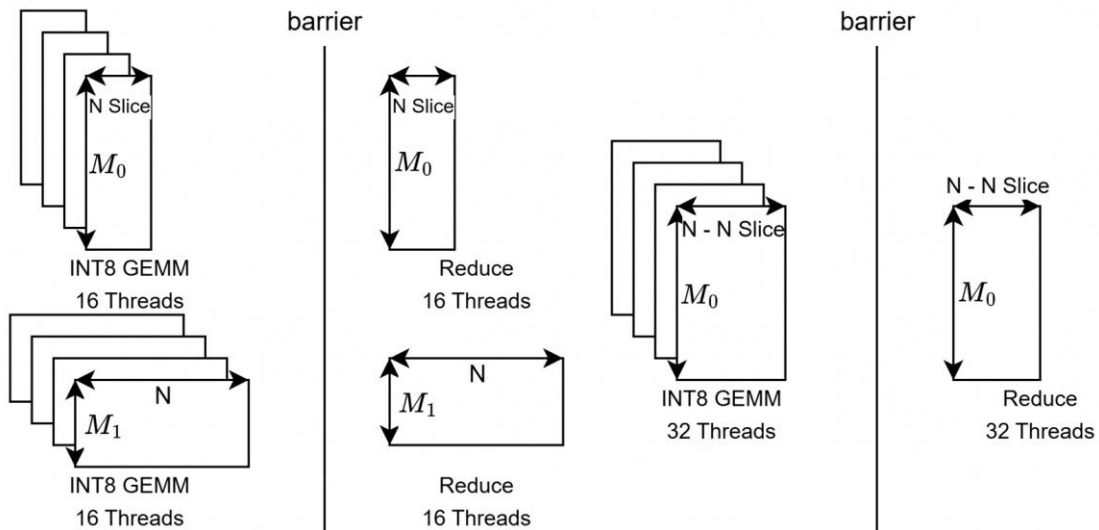
1.3.6 Dual Experts Parallel

在混合专家模块中每个专家接收到的 Token 数量各不相同，而在 DeepSeek 应用中，收到 SME 自身几何特性约束，DeepSeek 官方提供的 FusedKernel 在性能上无法等效适配 SME 指令集。因此对于当前推理过程中使用的双专家场景，需要设计双专家并行计算的方案，以增加线程子任务的计算量，进一步提高计算效率。



图表 13 当前混合专家门控 IGEMM 方案

对于上图所示的原混合专家串行计算方案，在计算效率上存在优化空间，由于专家持有 Token 数量是随机的，在双专家场景中，若两专家持有 Token 数量差距较大 $M_0 \gg M_1$ ，在计算上则表现为 Token 数少的专家 M_1 计算效率偏低。因此，DeepSeek 团队提出了基于 N Slice 的专家并行方案，旨在平衡双专家 Token 不均衡引起的计算效率问题，如下图：



图表 14 双专家并行方案

理论基础：在 SME 中，我们观察到，对于 $[M, N, K]$ IGEMM，M 在两个相邻 16 倍数之间的任意值对最终的计算延时几乎无影响。例如对于两个专家，分别接收 Token 数量 3 和 16，在 SME 指令集的计算过程中，由于单次 MOPA 的原子计算规模为 $[16, 16, 4]$ ，这导致两个专家事实上需要的 MOPA 指令延时一致，最终计算延时也几乎相同，因此在一定峰均比范围内，输入情况可以穷举。

调度策略：双专家并行方案通过将 Token 数多的专家 M_0 的计算目标在 N 维度切割为两部分，通过扫参实验确定当前场景下穷举输入对应的 N Slice 最佳值并构建 Map，以在计算延时上与 M_1 接近，并且两个任务均由原先线程数的一半执行，由此增加单线程子任务的计算量，使 SME IGEMM 避免由于矩阵规模过小导致的计算效率下降问题。

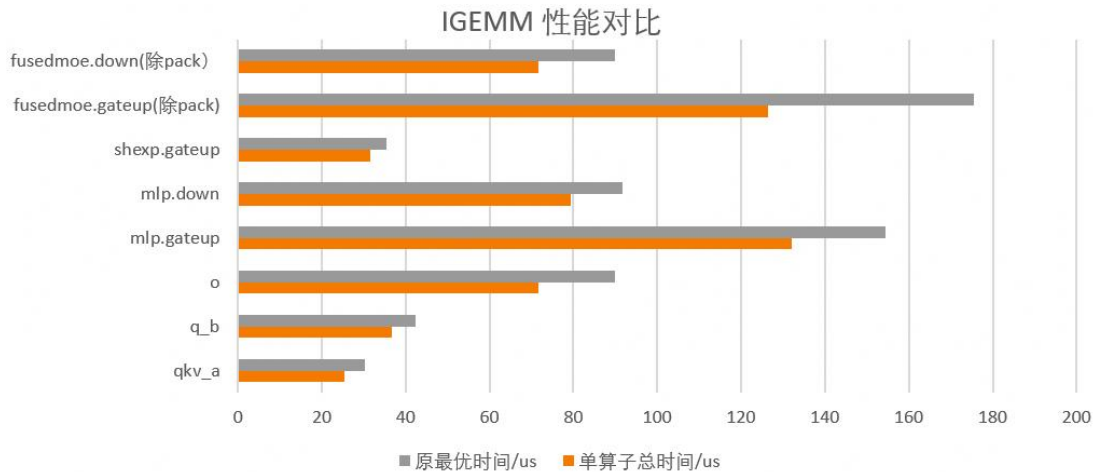
该方案可进一步拓展为 IGEMM 多任务并行，通过具体指定线程子任务的分配模式，可以使 IGEMM 快速适配混合专家类似的动态计算规模场景中的高性能运行。

局限性和注意事项：

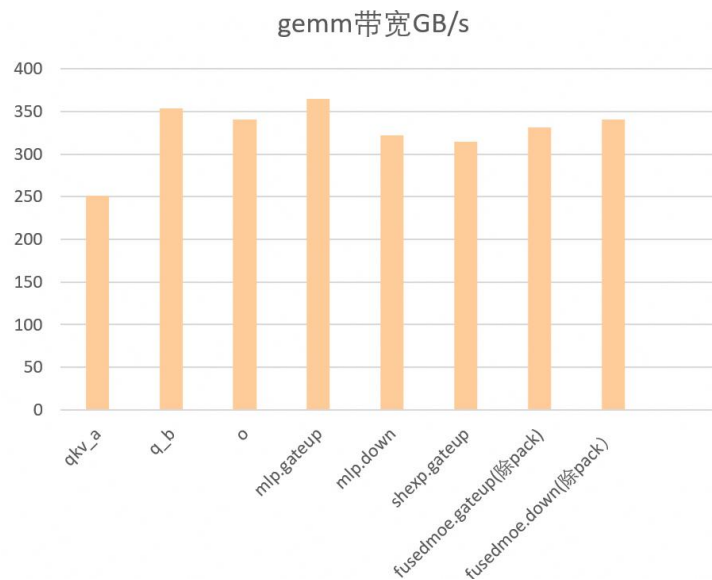
1. 相比较 GPU SIMT 硬件，ARM SME 指令集计算 GEMM 受到计算最小规模，非线性计算规模延时比，无零开销负载均衡机制三个方面约束。
 - a) 计算最小规模为 ZA Tile 单次 MOPA $[M, N, K] = [16, 16, 4]$ ，直接对任务切分粒度构成最小几何约束。
 - b) 非线性计算规模延时比，指相比较 GPU 在 K 一致的场景中，M, N 的变化和最终延时不成线性关系，难以进行规模延时建模，只能通过 Map/JIT 方法强制约束。
 - c) 无零开销负载均衡机制，意味着在 Decode 等计算规模有限且对延时敏感的场景中，多专家并行做实时负载均衡方案计算的过程本身，将掩盖并行计算带来的收益。简易负载均衡抽象可能延时将达到 10us+，不可接受。
2. 因此，面对 Decode 等严苛计算场景时，最佳方案为延时硬编码的无抽象手动负载均衡，通过如双专家方案的扫参查找 N Slice，可以最大程度上减小由于专家并行均衡抽象层和调度计算引发的性能问题。

1.4 IGEMM 系列算子优化性能

经由上述过程优化，DeepSeek 应用中 IGEMM 相关算子在优化的前后性能对比如下图所示，优化后的 IGEMM 性能相比较原有一般预取 IGEMM 有 10%~25% 的性能提升。



同时测量各个 IGEMM 算子 GEMM 阶段带宽，如下图所示，观察到多数算子带宽均在 300GB/s 以上，其 HBM 带宽利用率达到约 80%，认为 IGEMM 算子受 HBM 带宽约束实现了极致优化。



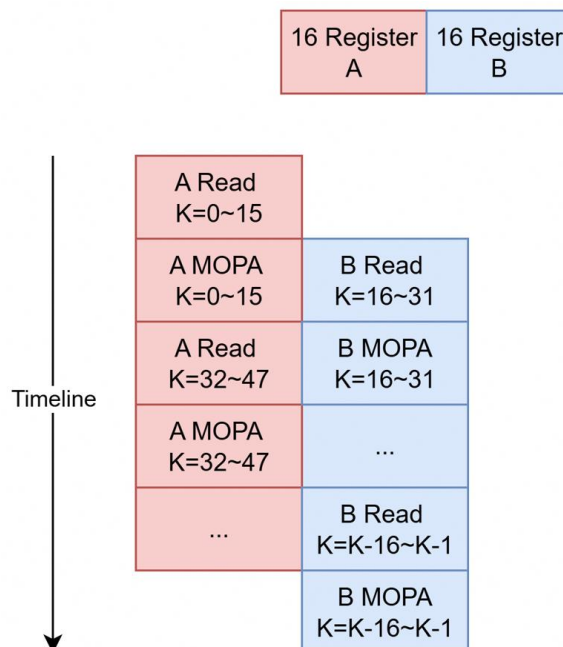
1.5 其他优化实验和分析

1.5.1 Pingpong 流水线实验

实验目的：在 Prefetch Pipeline Kernel 的基础上验证 IGEMM 在不同硬件约束下的性能上限。

实验操作：

在 Prefetch Pipeline Kernel 中，做 $2VL \times 2VL$ 的 MOPA 需要 4 个寄存器输入，循环展开四次后需要 16 个寄存器输入，在此基础上引入另外 16 个寄存器构建流水线，大概过程如下图：



图表 15 Pingpong 流水线示意图

通过如图所示的两组寄存器形成 Double Buffer，能够消除同一组寄存器连续 MOPA 和 READ 之间强依赖关系导致的 CPU Stall 问题。

同时，建立另一个组，其在 Prefetch Pipeline Kernel 的基础上，通过强制预取和等待保证计算窗口中左右矩阵输入的 L2 Cache HIT Rate 为 100%。

实验结果：

1. 只应用 Pingpong 的性能无提升，此时瓶颈仍然为内存带宽。
2. 在输入矩阵 100% L2 Cache HIT 的前提下，IGEMM 计算效率有最高 55% 提升到 70%，此时经分析，硬件约束瓶颈为 L2→L1 访存带宽。

实验推算：

1. 已知在只应用 Pingpong 的场景中 IGEMM 计算效率 55%时，带宽为 330GB/s，则 70%时带宽为 420GB/s，意味着 HBM 需再提升 30%才不会成为硬件性能的约束，此时 L2→L1 访存带宽计算：在 2.0GHz 下，计算效率为 70%，则一组 4 tile mopa 耗时 $\frac{4\text{cycle}}{70\%} = 5.7\text{cycle}$ ，在此期间完成了 L2→L1 访存 $\frac{4 \times 64\text{Byte}}{\text{Bandwidth}} = 5.7\text{cycle}$ ，求得 Bandwidth = 44GB/s。
2. 假设预期 IGEMM 要做到 90%，则此时对应 HBM 带宽为 540GB/s。此时 L2→L1 带宽需提高至 56GB/s。